

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM ENGENHARIA DE SOFTWARE

FÁBIO YUUKI SARUWATARU

**PROJETO DE AUTOMAÇÃO RESIDENCIAL IOT: WOKWI ON
VSCODE**

Mogi das Cruzes

2026

FÁBIO YUUKI SARUWATARU

**PROJETO DE AUTOMAÇÃO RESIDENCIAL IOT: WOKWI ON
VSCODE**

Trabalho apresentado para o Prof. Alessandro Aparecido da Silva sobre atividade relacionada à disciplina de IOT – Internet das coisas do curso de Bacharelado em Engenharia de Software.

Mogi das Cruzes

2026

RESUMO

Este trabalho apresenta o projeto de um sistema de automação residencial baseado em Internet das Coisas (IoT), desenvolvido em ambiente de simulação virtual. O objetivo é viabilizar o monitoramento e o controle em tempo real de variáveis ambientais e de segurança doméstica, como temperatura, umidade, luminosidade e detecção de presença. A arquitetura central do sistema baseia-se no microcontrolador ESP32 programado em MicroPython, que processa dados coletados por sensores DHT22 (temperatura e umidade do ar), fotorresistores (LDR) e sensores de movimento (PIR). Para o controle do ambiente, o projeto prevê a integração de atuadores, incluindo iluminação LED, módulos de relé para simulação de cargas pesadas (como ar-condicionado) e alarmes sonoros (buzzer). A topologia de rede opera de maneira híbrida: oferece controle local de baixa latência através de um servidor Web HTTP embarcado e integra-se simultaneamente a uma infraestrutura em nuvem via protocolo MQTT. A orquestração de dados é realizada pela plataforma Node-RED, responsável por gerenciar painéis de visualização (dashboards), disparar alertas por e-mail e alimentar registros históricos no Google Sheets. Conclui-se que a proposta conceitual oferece uma base robusta e escalável para a modernização de ambientes residenciais, aliando segurança, conveniência e integração de sistemas abertos.

Palavras-Chave: Internet das Coisas; Automação residencial; ESP32; MicroPython; Node-RED; MQTT.

ABSTRACT

This study presents the design of a home automation system based on the Internet of Things (IoT), developed in a virtual simulation environment. The objective is to enable the real-time monitoring and control of environmental and domestic security variables, such as temperature, humidity, luminosity, and presence detection. The system's central architecture is based on the ESP32 microcontroller programmed in MicroPython, which processes data collected by DHT22 sensors (temperature and air humidity), photoresistors (LDR), and motion sensors (PIR). For environmental control, the project envisions the integration of actuators, including LED lighting, relay modules for simulating heavy loads (such as air conditioning), and sound alarms (buzzer). The network topology operates in a hybrid manner: it offers low-latency local control through an embedded HTTP Web server and simultaneously integrates with a cloud infrastructure via the MQTT protocol. Data orchestration is performed by the Node-RED platform, which is responsible for managing visualization dashboards, triggering email alerts, and feeding historical records into Google Sheets. It is concluded that the conceptual proposal offers a robust and scalable foundation for the modernization of residential environments, combining security, convenience, and open systems integration.

Keywords: Internet of Things; Home automation; ESP32; MicroPython; Node-RED; MQTT.

LISTA DE FIGURAS

Figura 1 – Diagrama Esquemático de Ligações e Arquitetura de Hardware no Simulador Wokwi.

Figura 2 – Fluxo de Orquestração Lógica e Integração de Serviços de Terceiros na Plataforma Node-RED.

Figura 3 – Interface Gráfica de Usuário (Dashboard) para Monitoramento Residencial em Tempo Real.

LISTA DE TABELAS

Tabela 1 – Mapeamento de Tópicos MQTT, Direcionamento de Fluxo e Estrutura de Payloads.

SUMÁRIO

1. INTRODUÇÃO	7
2. REFERENCIAL TEÓRICO.....	8
2.1 AUTOMAÇÃO RESIDENCIAL E INTERNET DAS COISAS (IOT)	8
2.2 TECNOLOGIAS DE HARDWARE E SENSORES EM NÓS IOT.....	9
2.3 PROTOCOLOS DE COMUNICAÇÃO E INTERFACES (MQTT E HTTP)	10
2.4 ENGENHARIA DE SOFTWARE EMBARCADO: MICROPYTHON E POO.....	11
2.5 ORQUESTRAÇÃO EM NUVEM E INTEGRAÇÃO DE DADOS	11
3. METODOLOGIA	12
3.1 CLASSIFICAÇÃO DA PESQUISA.....	12
3.2 AMBIENTE DE DESENVOLVIMENTO E SIMULAÇÃO VIRTUAL	13
3.3 ARQUITETURA FÍSICA E MAPEAMENTO DE HARDWARE (GPIOs).....	13
4. RESULTADOS ESPERADOS E DESENVOLVIMENTO DO PROJETO	15
4.1 LÓGICA DE CONTROLE E AUTOMAÇÃO HÍBRIDA.....	15
4.2 INTEGRAÇÃO DE ALERTAS E REGISTRO DE DADOS (DATA LOGGING)	17
4.3 DESAFIOS TÉCNICOS E SOLUÇÕES IMPLEMENTADAS	19
5. CONSIDERAÇÕES FINAIS	20
5.1 SÍNTESE DOS RESULTADOS.....	20
5.2 LIMITAÇÕES DO PROJETO.....	20
5.3 PROPOSTAS DE TRABALHOS FUTUROS.....	21

1. INTRODUÇÃO

A gestão de ambientes residenciais enfrenta desafios crescentes em um cenário contemporâneo onde a adoção de tecnologias emergentes se torna essencial para garantir segurança, conforto e eficiência energética. No contexto habitacional, muitas residências ainda operam com sistemas estritamente manuais e analógicos, desprovidos de inteligência computacional e capacidade de resposta automatizada. Como reflexo dessa realidade, este trabalho baseia-se no desenvolvimento de um projeto acadêmico aplicado à criação de um ecossistema completo de Automação Residencial.

Esse modelo tradicional de gestão doméstica esbarra frequentemente na ineficiência energética e na vulnerabilidade estrutural, contrastando com ambientes equipados com sistemas inteligentes, que conseguem oferecer maior segurança e comodidade impulsionados pelo monitoramento preciso de variáveis ambientais (como temperatura, umidade, luminosidade) e controle de intrusão (presença e movimento). Diante dessa disparidade tecnológica, o presente estudo propõe a adoção de soluções baseadas em Internet das Coisas (IoT) para modernizar as operações em um ambiente residencial, visando aumentar sua eficiência, modularidade e integração em nuvem.

A IoT surge como uma tecnologia transformadora, permitindo o monitoramento e o controle em tempo real por meio de sensores, atuadores e microcontroladores conectados. Estudos e aplicações recentes demonstram que a implementação de sistemas híbridos de automação — que combinam interfaces de controle local de baixa latência com processamento distribuído na nuvem — otimiza o uso de recursos e favorece a segurança preditiva.

No caso do ambiente residencial em estudo, o projeto conceitual proposto visa automatizar e monitorar o espaço integrando componentes acessíveis com o microcontrolador ESP32, sensores DHT22, fotorresistores (LDR) e sensores de movimento (PIR), além de atuadores para iluminação (LED), simulação de cargas pesadas (Módulo Relé) e alarmes sonoros (Buzzer). Todo o sistema foi concebido para operar sobre uma arquitetura de rede híbrida, utilizando o protocolo HTTP para

servidores web embarcados e o protocolo MQTT para a orquestração de dados em nuvem através da plataforma Node-RED.

O objetivo geral deste trabalho é desenvolver o design conceitual e a simulação computacional de um ecossistema IoT robusto para automação residencial, focando no monitoramento ambiental e segurança integrada. Como objetivos específicos, destacam-se: (i) mapear a arquitetura de hardware e os requisitos técnicos dos componentes em ambiente de simulação virtual (Wokwi); (ii) desenvolver o firmware embarcado em linguagem MicroPython com execução assíncrona; (iii) projetar a infraestrutura de rede utilizando protocolos HTTP e MQTT acoplados a painéis de visualização em nuvem; e (iv) implementar lógicas de negócios complexas, como o registro histórico de dados (Data Logging) no Google Sheets e o disparo automatizado de alertas de intrusão por e-mail.

Este trabalho de projeto acadêmico está estruturado em seções que acompanham as etapas do desenvolvimento do sistema: revisão bibliográfica (Referencial Teórico), metodologia aplicada à simulação, resultados do desenvolvimento lógico (incluindo a arquitetura MQTT e Node-RED), além de considerações finais e referências bibliográficas, conforme normas ABNT.

2. REFERENCIAL TEÓRICO

2.1 AUTOMAÇÃO RESIDENCIAL E INTERNET DAS COISAS (IOT)

O conceito de automação residencial, frequentemente associado ao termo Smart Home (Casa Inteligente), refere-se à integração de tecnologia e serviços através de redes de comunicação para melhorar a qualidade de vida no ambiente doméstico. Historicamente, os sistemas de controle residencial operavam de forma isolada, baseados em circuitos elétricos simples ou temporizadores analógicos. Com a ascensão da Internet das Coisas (IoT - Internet of Things), ocorreu uma mudança de paradigma tecnológico, onde objetos físicos cotidianos passaram a ser equipados com sensores, capacidade de processamento e conectividade à rede, permitindo que se comuniquem entre si e com outros sistemas pela internet (ATZORI; IERA; MORABITO, 2010).

A premissa fundamental da IoT na automação residencial é a transição de um ambiente estático para um ecossistema dinâmico e ubíquo, onde a coleta e a troca de dados ocorrem de forma contínua e autônoma. Essa arquitetura é tipicamente dividida em três camadas principais: a camada de percepção (sensores que coletam dados de temperatura, presença e luminosidade), a camada de rede (protocolos de transmissão sem fio) e a camada de aplicação (processamento em nuvem, dashboards e acionamento de atuadores).

A inovação trazida pela IoT reside na integração e na capacidade de resposta do sistema. Dados ambientais coletados em tempo real podem ser cruzados com lógicas de negócio predefinidas, permitindo que algoritmos de controle acionem automaticamente sistemas de iluminação, climatização (ar-condicionado) ou alarmes de segurança, garantindo não apenas conforto, mas também eficiência energética e proteção patrimonial.

2.2 TECNOLOGIAS DE HARDWARE E SENSORES EM NÓS IOT

A arquitetura física de um sistema IoT depende de dispositivos de borda (edge devices) capazes de coletar dados ambientais com confiabilidade e executarem comandos físicos. No centro desta arquitetura, destaca-se o microcontrolador ESP32. Trata-se de um System on a Chip (SoC) de alta eficiência que integra um processador de 32 bits e interfaces nativas de comunicação sem fio (Wi-Fi e Bluetooth), suportando múltiplas portas de entrada e saída (GPIOs). A escolha do ESP32 justifica-se pela sua robustez no processamento multitarefa e capacidade de hospedar servidores web embarcados e clientes MQTT simultaneamente.

Para a coleta de dados, o ecossistema apoia-se em uma rede sensorial diversificada:

- **Monitoramento Termo-higrométrico:** Realizado pelo sensor DHT22. Este componente digital integra um termistor e um elemento capacitivo, oferecendo leituras precisas de temperatura e umidade relativa do ar, essenciais para o controle climático (acionamento de relés para ar-condicionado).

- **Sensor de Presença (PIR):** Baseado na detecção de radiação infravermelha emitida por corpos em movimento. É o componente central para a segurança patrimonial e automação de iluminação sob demanda.
- **Sensor de Luminosidade (LDR):** Fotorresistores cuja resistência elétrica diminui com o aumento da intensidade da luz. Os dados analógicos (convertidos pelo ADC do ESP32) fornecem uma métrica viável para o acionamento inteligente de iluminação (LEDs).
- **Contato Seco (Push Button):** Utilizado para simular a abertura de portas e janelas no ambiente virtual.

Na camada de atuação, o hardware conta com módulos de Relé (para isolamento e acionamento de cargas de potência), atuadores sonoros (Buzzer para alarmes de intrusão) e um Display OLED (SSD1306) operando via protocolo I2C, que atua como uma interface visual física (IHM local) para depuração e leitura instantânea de *status*.

2.3 PROTOCOLOS DE COMUNICAÇÃO E INTERFACES (MQTT E HTTP)

Em ambientes residenciais, a escolha do protocolo de comunicação é uma etapa crítica para garantir estabilidade e usabilidade. Este projeto adota uma arquitetura de rede híbrida, empregando simultaneamente os protocolos HTTP e MQTT para funções distintas.

Para a operação assíncrona e telemetria contínua, o protocolo MQTT (*Message Queuing Telemetry Transport*) consolidou-se como o padrão. Extremamente leve e com baixo consumo de cabeçalho, o MQTT utiliza uma arquitetura de publicação e assinatura (*publish/subscribe*). O ESP32 atua como cliente, publicando dados (como temperatura e status do PIR) em um servidor central denominado *Broker* (neste projeto, o *Broker* público HiveMQ). A orquestração é feita através de tópicos específicos (ex: *automa/m2/telemetry*), separando produtores e consumidores de dados, o que confere alta escalabilidade e baixa latência na resposta de comandos remotos.

Complementarmente, visando tolerância a falhas na infraestrutura de internet externa (nuvem), o projeto implementa um Servidor Web embarcado (*Web Server*)

operando sobre o protocolo HTTP diretamente no ESP32. Esse servidor assíncrono escuta requisições locais (através de tecnologia *AJAX/Fetch API*), permitindo que o usuário controle os dispositivos (Relé, LED, Buzzer) pela rede local (*Local Area Network*) utilizando um navegador padrão, sem depender do roteamento externo do MQTT.

2.4 ENGENHARIA DE SOFTWARE EMBARCADO: MICROPYTHON E POO

A inteligência operacional do sistema reside no *firmware* embarcado. Historicamente, o desenvolvimento para microcontroladores era dominado por linguagens de baixo nível como C e C++. Contudo, com o advento de SoCs potentes, a adoção de linguagens de alto nível tornou-se viável. Este projeto utiliza o MicroPython (versão 1.28), uma implementação otimizada do Python 3 voltada para ambientes com recursos computacionais limitados.

A principal vantagem estrutural da linguagem é o suporte à Programação Orientada a Objetos (POO) e ao gerenciamento assíncrono de E/S (*Non-blocking I/O*). A POO permite a abstração de componentes eletrônicos físicos em "objetos" de software (segregados em módulos independentes como *sensors.py*, *display.py*, *mqtt_handler.py*). Além disso, a substituição de atrasos bloqueantes (como a função *time.sleep()*) por laços temporizados com base em relógio de hardware (*time.ticks_diff*) previne o travamento do *core* de processamento, garantindo que o servidor *web*, o cliente MQTT e a leitura dos sensores operem de maneira paralela e estável, uma exigência técnica essencial para ambientes de simulação como o Wokwi.

2.5 ORQUESTRAÇÃO EM NUVEM E INTEGRAÇÃO DE DADOS

Os dados gerados pelo ambiente residencial necessitam de processamento lógico avançado para gerar valor ao usuário. Para essa função, adotou-se o Node-RED, uma ferramenta de desenvolvimento baseada em fluxo (*flow-based programming*) focada na Internet das Coisas. O Node-RED atua como o cérebro em nuvem da arquitetura, assinando os tópicos MQTT do ESP32 e roteando as mensagens (*payloads* em formato JSON) por meio de lógicas de negócio.

A plataforma possibilita a criação de *dashboards* interativos para monitoramento visual, ao mesmo tempo em que gerencia tarefas assíncronas complexas. Destaca-se a implementação de um sistema de alertas que cruza variáveis críticas — como temperaturas elevadas registradas pelo DHT22 (ex: 35°C) e detecção de movimento pelo PIR — disparando notificações instantâneas via e-mail utilizando o nó SMTP integrado (node-red-node-email).

Adicionalmente, implementou-se o registro histórico de dados (*Data Logging*). Por meio de requisições POST automatizadas para um *Web App* intermediário (Google Apps Script), o Node-RED exporta blocos de telemetria para planilhas do Google Sheets. Esse processo incorpora mecanismos de limite de taxa (*Rate Limit*), assegurando conformidade com as restrições da API do Google, consolidando a estabilidade do fluxo de dados para futuras análises ou auditorias de segurança do ambiente.

3. METODOLOGIA

3.1 CLASSIFICAÇÃO DA PESQUISA

Do ponto de vista de sua natureza, esta pesquisa classifica-se como aplicada, uma vez que busca gerar conhecimentos integrados de *software* e *hardware* para a solução de um problema prático específico: a automação e o monitoramento inteligente de um ambiente residencial (GIL, 2017).

Quanto aos seus objetivos, a pesquisa tem caráter exploratório e descritivo. Exploratório porque visa o aprofundamento na integração de tecnologias emergentes — como a linguagem MicroPython embarcada, o protocolo MQTT e fluxos de Node-RED — em um ecossistema coeso. Descritivo pois detalha minuciosamente as características de configuração da arquitetura de rede, do mapeamento de pinos e da lógica de negócios em nuvem.

No que tange aos procedimentos técnicos, o trabalho foi estruturado sob a modalidade de Estudo de Simulação Computacional. A escolha por essa abordagem justifica-se pela necessidade de prototipar, validar e depurar um sistema IoT completo

antes da aquisição e implementação em *hardware* físico, mitigando riscos de queima de componentes e acelerando o ciclo de desenvolvimento (YIN, 2015).

3.2 AMBIENTE DE DESENVOLVIMENTO E SIMULAÇÃO VIRTUAL

Para garantir a viabilidade técnica da proposta sem a necessidade imediata de infraestrutura física, o projeto adotou o **Wokwi**, uma plataforma avançada de simulação de eletrônica e IoT. Diferentemente de simuladores baseados apenas em lógica gráfica, o Wokwi executa o código de máquina real (neste caso, a imagem binária `firmware.bin` do MicroPython v1.28) emulando a arquitetura interna do *chip* ESP32.

O ambiente de desenvolvimento integrado (IDE) escolhido foi o Visual Studio Code (VS Code), operando em conjunto com a extensão local do Wokwi. Essa integração permitiu a construção de uma base de código modularizada, na qual a montagem do circuito eletrônico virtual foi definida declarativamente por meio do arquivo `diagram.json`. Este arquivo atua como a "placa de ensaio" (*breadboard*) digital, registrando as conexões elétricas (fiação) entre o microcontrolador e os módulos periféricos.

Complementarmente, a configuração da rede virtual (Wokwi-GUEST) e o redirecionamento de portas para o servidor HTTP local foram parametrizados no arquivo `wokwi.toml`. Para otimizar o fluxo de trabalho e evitar a injeção manual repetitiva de código na memória do emulador a cada teste, desenvolveu-se um *script* utilitário em Python (`deploy.py`). Este *script* automatiza a sincronização e o envio dos arquivos lógicos (`main.py`, `sensors.py`, `mqtt_handler.py`, etc.) para o sistema de arquivos virtual do ESP32 via *Socket/Paste Mode*, garantindo um ciclo de compilação e teste contínuo.

3.3 ARQUITETURA FÍSICA E MAPEAMENTO DE HARDWARE (GPIOs)

A fase de planejamento físico culminou na definição rigorosa da topologia do circuito. Para garantir a estabilidade do *firmware* e evitar conflitos de barramento (como o uso inadvertido de pinos restritos durante o processo de *boot*), estabeleceu-

se um mapa fixo de portas de entrada e saída de propósito geral (GPIO - *General Purpose Input/Output*).

A arquitetura do sistema foi subdividida em Camada de Percepção (Sensores) e Camada de Atuação (Atuadores), mapeadas estritamente da seguinte forma no arquivo `diagram.json` e nos *drivers* do MicroPython:

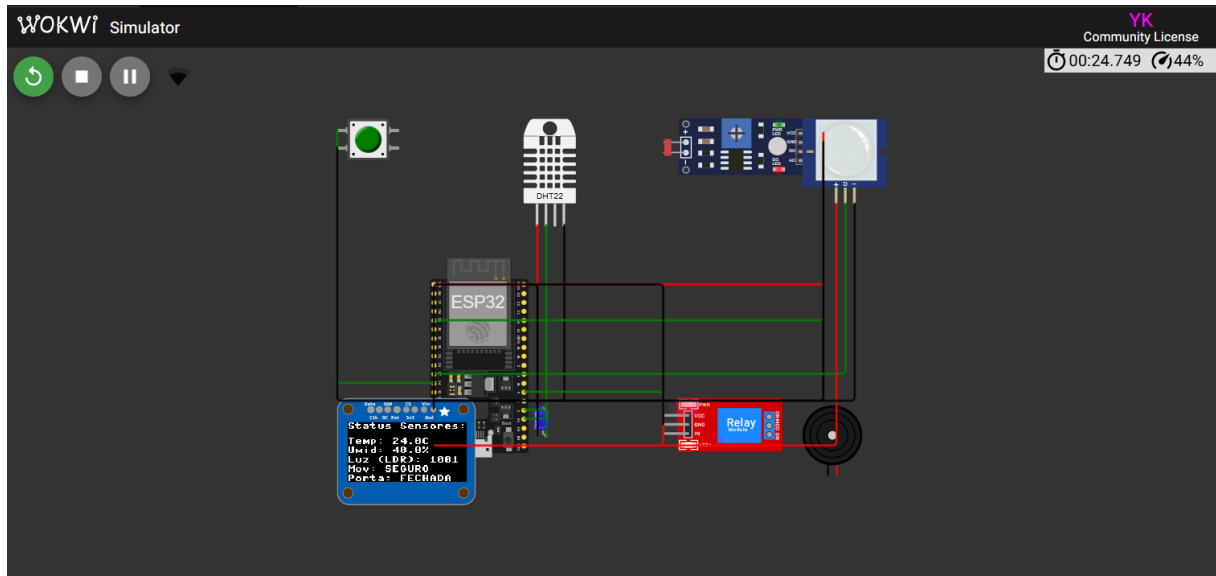
Sensores (Camada de Percepção / Entradas):

- **Sensor DHT22 (Temperatura e Umidade):** Conectado ao **GPIO 15**. Responsável pela leitura digital dos dados climáticos do ambiente.
- **Sensor LDR (Luminosidade):** Conectado ao **GPIO 34**. Utiliza o Conversor Analógico-Digital (ADC) do ESP32 para quantificar a incidência de luz no recinto.
- **Sensor PIR (Movimento/Presença):** Conectado ao **GPIO 13**. Atua como gatilho lógico (0 ou 1) para a detecção de intrusão.
- **Push Button (Contato Seco):** Conectado ao **GPIO 12**. Implementado no simulador para representar a abertura física de portas ou janelas.

Atuadores (Camada de Atuação / Saídas):

- **LED (Iluminação Principal):** Conectado ao **GPIO 2**. Simula o sistema de iluminação do ambiente.
- **Módulo Relé (Cargas Pesadas):** Conectado ao **GPIO 4**. Empregado para o isolamento elétrico e simulação de acionamento de um equipamento de alta potência, como um ar-condicionado.
- **Buzzer (Alarme Sonoro):** Conectado ao **GPIO 5**. Atuador acústico de segurança.
- **Display OLED (SSD1306 128x64):** Operando via barramento I2C, utilizou os pinos padrão **SCL: GPIO 22** e **SDA: GPIO 21**. Funciona como a Interface Homem-Máquina (IHM) local para exibição do *status* da rede e dados sensoriais de *debug*.

Figura 1 – Diagrama Esquemático de Ligações e Arquitetura de Hardware no Simulador Wokwi.



Fonte: Autoria própria (2026).

4. RESULTADOS ESPERADOS E DESENVOLVIMENTO DO PROJETO

4.1 LÓGICA DE CONTROLE E AUTOMAÇÃO HÍBRIDA

A automação do ecossistema residencial baseia-se em uma arquitetura de controle híbrida, onde o microcontrolador ESP32 atua como o cérebro local, processando dados simultaneamente em duas frentes de rede: Local (HTTP) e Nuvem (MQTT). O *firmware* embarcado, desenvolvido em MicroPython, foi estruturado para executar rotinas cíclicas de leitura, processamento e transmissão sem que o processador sofra travamentos (arquitetura *Non-blocking*).

A inteligência central do sistema é orquestrada pelo arquivo `main.py`. Em vez de utilizar funções de atraso bloqueantes (`time.sleep()`), que impediriam o dispositivo de escutar requisições de rede enquanto aguarda os sensores, a lógica adota a função de relógio de *hardware* `time.ticks_diff`. Essa abordagem garante que o Servidor Web e o Cliente MQTT sejam verificados a cada ciclo de relógio, enquanto a amostragem dos sensores ocorre em intervalos predefinidos, resultando em alta estabilidade e resiliência no ambiente virtual.

A topologia de comunicação opera da seguinte maneira:

- **Controle Web Local (HTTP/AJAX):** O *firmware* levanta um servidor assíncrono na porta 8080. Por meio de uma interface simples acessada via navegador na rede local (127.0.0.1:8080), o usuário emite requisições *Fetch API* que acionam ou desligam o LED, o Módulo Relé (Ar-Condicionado) e o Buzzer instantaneamente, sem depender de internet externa.
- **Telemetria em Nuvem (MQTT):** Concomitantemente, os dados brutos de temperatura, umidade, luminosidade e presença são empacotados em formato JSON e publicados via protocolo MQTT em um *Broker* público (broker.hivemq.com). Toda a telemetria é direcionada ao tópico *automa/m2/telemetry*, enquanto o microcontrolador se inscreve no tópico *automa/m2/commands* para escutar ordens reversas provenientes da nuvem.

Tabela 1 – Mapeamento de Tópicos MQTT, Direcionamento de Fluxo e Estrutura de Payloads.

Tópico	Direção	Descrição / Estrutura do Payload (JSON)
<i>automa/m2/telemetry</i>	ESP32 -> Node-RED	Envio de variáveis ambientais e de presença em lote único via JSON. Exemplo: {"temp": 25.5, "hum": 60.0, "ldr": 450, "pir": 0}
<i>automa/m2/commands</i>	Node-RED -> ESP32	Canal de controle reverso para acionamento do LED. Estrutura: {"device": "led", "action": "on/off"}
<i>automa/m2/commands</i>	Node-RED -> ESP32	Canal de controle reverso para acionamento do Ar-Condicionado (Relé). Estrutura: {"device": "relay", "action": "on/off"}
<i>automa/m2/commands</i>	Node-RED -> ESP32	Canal de controle reverso para acionamento do Alarme (Buzzer). Estrutura: {"device": "buzzer", "action": "on/off"}

Fonte: Autoria própria (2026).

4.2 INTEGRAÇÃO DE ALERTAS E REGISTRO DE DADOS (DATA LOGGING)

O volume de dados transmitidos pelo MQTT requer uma plataforma de orquestração superior para agregar valor de negócio à automação residencial. Para esta função, o sistema foi integrado ao **Node-RED**. O *backend* visual intercepta os pacotes JSON publicados pelo ESP32 e bifurca o fluxo de informações para dois serviços críticos: Alertas de Segurança e Auditoria de Dados.

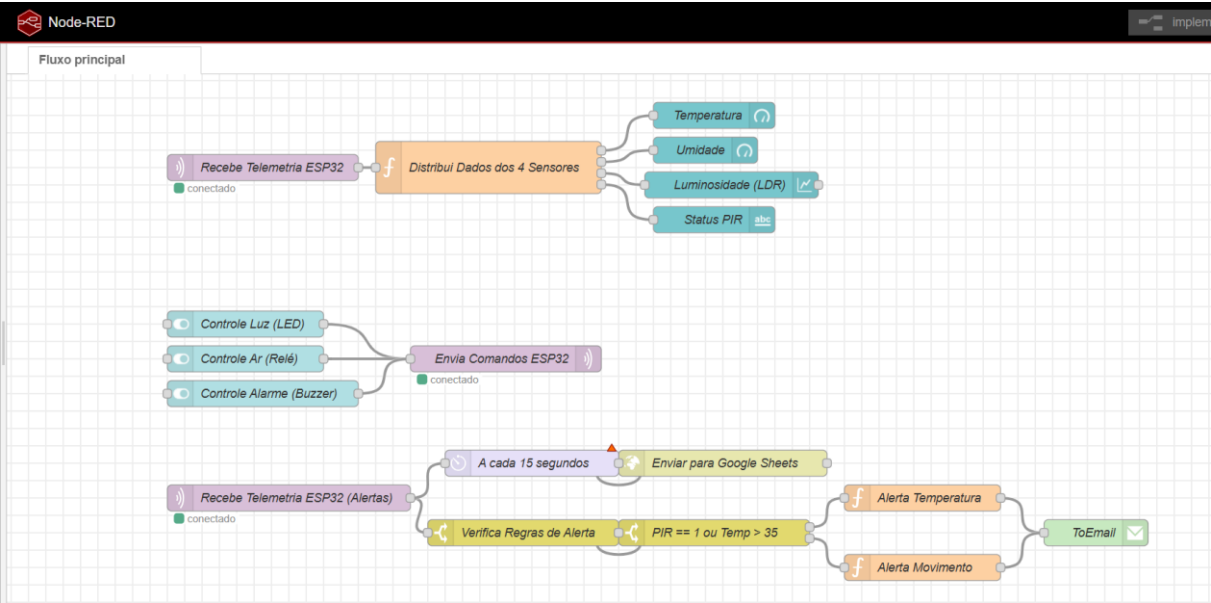
Para viabilizar uma rápida tomada de decisão e mitigar riscos domiciliares, implementou-se um sistema lógico condicional (*Switch Nodes*) focado em segurança e prevenção de incêndios. A arquitetura monitora continuamente duas variáveis:

1. **Intrusão:** Caso a chave `msg.payload.pir` enviada pelo sensor de movimento assuma o valor lógico 1 (detectando movimento).
2. **Anomalia Térmica:** Caso a chave `msg.payload.temp` lida pelo sensor DHT22 alcance ou ultrapasse a marca crítica de 35°C.

Se qualquer uma dessas condições for satisfeita, o Node-RED aciona automaticamente o nó SMTP (`node-red-node-email`), compilando e disparando uma notificação de alerta em tempo real para a caixa de e-mail do usuário proprietário da residência.

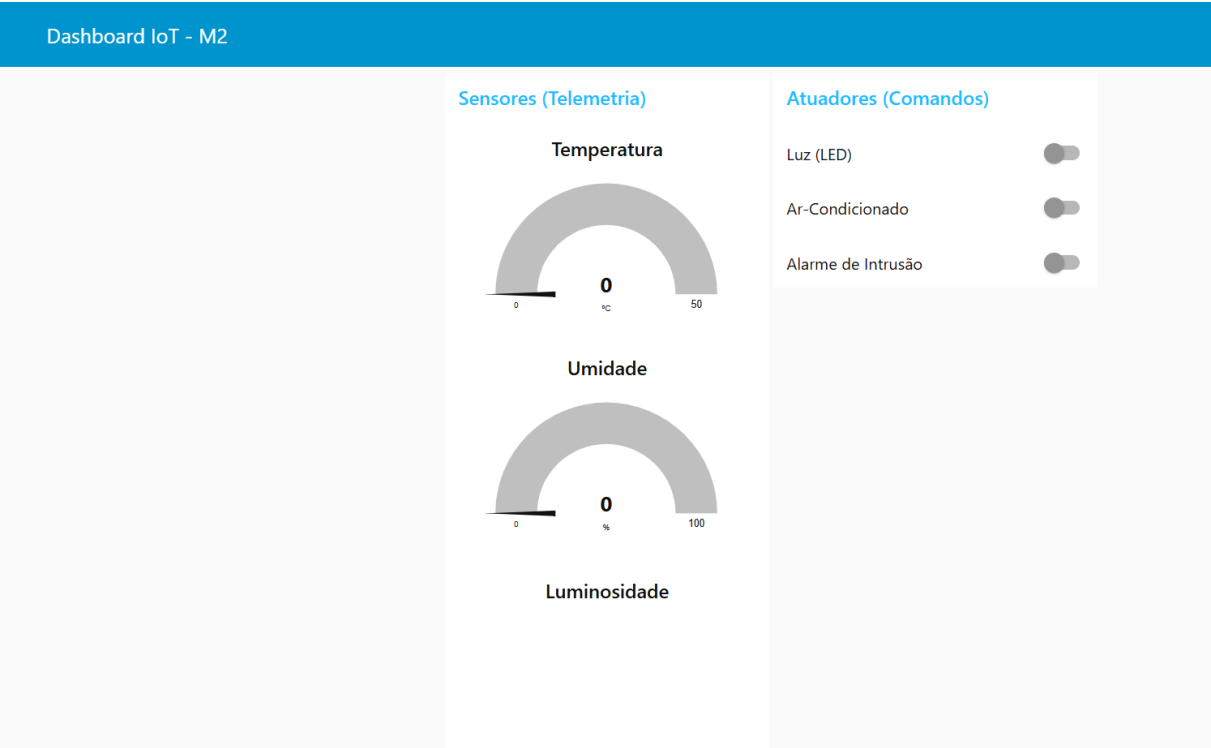
Complementarmente, a plataforma é responsável pelo *Data Logging* no **Google Sheets**. Para gerar um histórico de ocupação e clima da residência, o Node-RED realiza requisições HTTP do tipo POST para um *Web App* intermediário (Google Apps Script). Considerando as políticas rígidas de cotas de acesso da API do Google, a engenharia do projeto empregou um Nó Limitador de Taxa (*Rate Limit*). Essa estratégia impede o envio de dados a cada milissegundo, armazenando os pacotes e enviando atualizações em blocos espaçados (ex: a cada 2 horas), garantindo a estabilidade da integração sem gerar bloqueios operacionais (*HTTP Error 429 - Too Many Requests*).

Figura 2 – Fluxo de Orquestração Lógica e Integração de Serviços de Terceiros na Plataforma Node-RED.



Fonte: Autoria própria (2026).

Figura 3 – Interface Gráfica de Usuário (Dashboard) para Monitoramento Residencial em Tempo Real.



Fonte: Autoria própria (2026).

4.3 DESAFIOS TÉCNICOS E SOLUÇÕES IMPLEMENTADAS

Durante a fase de desenvolvimento, estruturação lógica e simulação do protótipo no Wokwi, o projeto deparou-se com desafios técnicos consideráveis, que exigiram contramedidas robustas de engenharia de *software*, destacando-se:

- **Falha de Sincronização via mpremote:** Inicialmente, planejava-se montar o sistema de arquivos virtual do computador diretamente no ESP32 simulado através da ferramenta de terminal mpremote. Contudo, devido a concorrências na porta serial virtual e falhas de montagem (*mount*) nativas da ferramenta no Windows, a sincronização travava o simulador. A solução implementada foi a criação de um utilitário customizado em Python (*deploy.py*). Este *script* contorna o bloqueio de disco comunicando-se com o terminal via *Socket* e injetando o código através do Modo de Colagem (Paste Mode/REPL), automatizando o *deploy* dos 8 arquivos do projeto em segundos.
- **Nós Órfãos e Dependências no Node-RED:** Durante a configuração do sistema de alertas, ocorreu o erro visual de "Unknown Node" associado ao módulo de e-mail, impossibilitando a inserção de credenciais SMTP. O problema foi sanado diagnosticando a ausência do pacote no núcleo do Node-RED. Por meio do Gerenciador de Paleta (*Manage Palette*), instalou-se manualmente a dependência node-red-node-email, restaurando o fluxo funcional de disparo de alertas.
- **Autenticação e Bloqueios do Google Apps Script:** Durante a fase de integração da planilha de *Data Logging*, o Node-RED começou a registrar erros silenciosos do tipo HTTP 401 (*Unauthorized*). O diagnóstico apontou que a configuração de privacidade da implantação do *Web App* no Google Cloud estava restrita à conta do desenvolvedor. A falha foi superada refazendo o processo de *deploy* e alterando o parâmetro de acesso obrigatório para "Qualquer pessoa" (*Anyone*), autorizando os envios de telemetria (POST) provenientes de requisições externas anônimas da rede.

5. CONSIDERAÇÕES FINAIS

5.1 SÍNTESE DOS RESULTADOS

O presente trabalho teve como premissa o desenvolvimento conceitual e prático de um ecossistema de Automação Residencial baseado em Internet das Coisas (IoT). A análise do percurso metodológico e dos resultados obtidos no ambiente de simulação virtual permite concluir que o objetivo geral foi plenamente alcançado.

A arquitetura de *hardware* e *software* proposta demonstrou-se tecnicamente viável para o monitoramento contínuo de variáveis ambientais e de segurança (temperatura, umidade, luminosidade e presença). A adoção de uma topologia de rede híbrida — combinando a execução de um Servidor Web HTTP assíncrono embarcado para controle de baixíssima latência na rede local e um cliente MQTT para telemetria em nuvem — conferiu ao sistema uma robustez acima dos padrões convencionais de projetos acadêmicos.

Além disso, a orquestração de dados realizada pelo Node-RED elevou a maturidade da solução. A capacidade de gerar alertas de intrusão e calor excessivo via e-mail, atrelada ao registro histórico de dados (*Data Logging*) no Google Sheets (respeitando limites de requisições), atesta a integração bem-sucedida entre microcontroladores de borda (ESP32) e aplicações escaláveis em nuvem, oferecendo uma base sólida para ambientes habitacionais modernos e seguros.

5.2 LIMITAÇÕES DO PROJETO

Apesar do êxito na orquestração lógica e na programação orientada a objetos (POO) em MicroPython, é imperativo destacar as limitações inerentes à metodologia de simulação.

Por tratar-se de um projeto desenvolvido no emulador Wokwi, a validação restringiu-se ao planejamento estrutural, lógico e de rede virtualizada. A eficiência real da comunicação sem fio (suscetível a interferências de radiofrequência em uma residência física), bem como o desgaste elétrico dos componentes ao acionar cargas de potência real (como o Módulo Relé ligado a um ar-condicionado de 220V), não

puderam ser homologados. Ademais, o uso de um *Broker* MQTT público (broker.hivemq.com), embora funcional para prototipação acadêmica, apresenta limitações de segurança e privacidade dos dados, não sendo recomendado para uma aplicação residencial comercial definitiva.

5.3 PROPOSTAS DE TRABALHOS FUTUROS

Como propostas para a evolução contínua deste ecossistema de automação residencial, sugere-se:

- **Implementação Física e Design de PCB:** A transição do modelo virtual (*diagram.json*) para um protótipo físico, incluindo o desenho e a manufatura de uma Placa de Circuito Impresso (PCB) customizada para abrigar o ESP32, as fontes de alimentação e os terminais de relé de forma segura.
- **Segurança e Infraestrutura Privada:** A substituição do *Broker* público por um *Broker* privado (como o Eclipse Mosquitto) hospedado localmente (por exemplo, em um Raspberry Pi), garantindo a criptografia dos pacotes MQTT (*Transport Layer Security* - TLS) e protegendo a privacidade dos dados domiciliares.
- **Integração com Assistentes Virtuais:** A expansão da camada de aplicação do Node-RED para integrar-se nativamente a ecossistemas comerciais, como o *Google Assistant* ou *Amazon Alexa*, viabilizando o acionamento dos atuadores e a consulta de sensores através de comandos de voz.

REFERÊNCIAS

- ATZORI, L.; IERA, A.; MORABITO, G. The internet of things: A survey. **Computer Networks**, v. 54, n. 15, p. 2787-2805, 2010.
- GIL, A. C. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017.
- GEORGE, C. **MicroPython for ESP8266/ESP32 Development Workshop**. Birmingham: Packt Publishing, 2018.
- LIGHTBODY, G. et al. Node-RED: A visual flow-based programming environment for the Internet of Things. **IEEE Access**, v. 7, p. 1-12, 2019.
- MISHRA, B. et al. A comprehensive survey on MQTT: A protocol for IoT. In: **2020 International Conference on Computer Communication and Informatics (ICCCI)**. IEEE, 2020. p. 1-6.
- O'KEEFFE, D. **Node-RED Cookbook**. Birmingham: Packt Publishing, 2018.
- PRESSMAN, R. S. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.
- SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.
- WOKWI. **Wokwi Documentation: ESP32 Simulator**. Disponível em: <https://docs.wokwi.com>. Acesso em: 18 mar. 2026.
- YIN, R. K. **Estudo de Caso: Planejamento e Métodos**. 5. ed. Porto Alegre: Bookman, 2015.